

从 Infra 角度看 为什么去掉 Encoder 更好

在过去半年，我研究上的主要叙事成为了尽可能去除 Inductive Bias，我也跟随 @Haiwen 成为了 Encoder-Free 的忠实拥趸者。

目前大多数多模态大模型（MLLM）都采用 LLaVA 架构：先由 SigLIP 一类的 Vision Encoder 处理图像，再用 Projector 将视觉特征映射到 LLM 的表示空间，最后与文本 token 合并后送入 LLM。

这种设计效果很好，LLaVA 几乎是引爆了多模态的热潮（最骄傲的信仰...），但在大规模训练中，它带来的系统成本不只是多了一个几百 M 参数的 ViT。更重要的是，它在原本相对规则的 LLM 训练图中，插入了一段计算规模、输入形状和并行策略都不完全相同的计算路径。

PS 本文主要讨论需要端到端 online 执行 Vision Encoder 的训练场景。如果 Encoder 完全冻结，且视觉特征可以离线预计算，下面的一些问题会明显减弱。

1. Vision Encoder 改变了原本规则的计算图

纯文本 LLM 的主干由一系列结构相似的 Transformer Layer 组成：

```
Tokens → Embedding → Transformer Layer 1 → ... → Transformer Layer N
```

纯文本训练当然也不是完全这里理想的，packing、MoE routing 都会抖，但至少 Transformer 主干本身足够规整，因此更容易建立稳定的成本模型，并组合 DP、TP、PP、CP、EP 等并行策略。

Encoder-based MLLM 则多了一条视觉路径：

```
Text IDs → Embedding -----\
                                   +→ Merge → LLM → Loss
Pixels → ViT → Projector -----/
```

其中，ViT 的输入是由图像 Patchify 得到的视觉 token。

ViT 和 LLM 并非两种完全无关的计算，二者通常都以 Transformer Block 为主，但在深度、hidden size、序列长度、输入形状和最佳并行配置上可能差异很大。因此，它们虽然属于同一个 end-to-end model，却不一定适合使用同一套执行方式。

Projector 不是这篇文章的重点。在采用浅层 MLP Projector 的常见架构中，它的计算量通常远小于 Vision Encoder 和 LLM；如果使用 Q-Former 等更复杂的 Connector，则需要另行评估，不过目前应该也没有多少人再使用了。

2. 多模态样本的计算成本更难预测

在本文讨论的动态分辨率 Encoder 中，输入图像的分辨率、图片数量和视频帧数会直接影响视觉 token 数量。例如：

```
Sample A: pure text
Sample B: one 448×448 image
Sample C: eight high-resolution images
Sample D: a long video
```

这些样本的计算成本至少包含两部分：

```
Cost(sample)
  approx. C_encoder(N_encoder_visual_tokens)
        + C_llm(N_text_tokens + N_llm_visual_tokens)
```

其中， $N_{\text{encoder_visual_tokens}}$ 是进入 Vision Encoder 的 patch/frame token 数， $N_{\text{llm_visual_tokens}}$ 是经过 Projector 或 token merge 后送入 LLM 的视觉 token 数。

这会产生两类负载不均衡：

- 不同 Data Parallel rank 拿到的图像分辨率、图片数量或视频长度不同，同步时需要等待最慢的 rank。
- 不同 micro-batch 的视觉负载不同，会让 Pipeline stage 的 service time 随时间波动。

如果训练数据同时包含纯文本、单图、多图和视频，执行路径还会随样本类型变化：纯文本样本不需要执行 Vision Encoder，视频样本则可能带来远高于平均值的视觉计算。

实际系统可以通过一些手段减小波动，但这也意味着 sampler 和 scheduler 必须理解模态信息。只统计 batch size 或文本 token 数，无法准确描述真实 workload；即使统计了进入 LLM 的全部 token，也还需要加上 Encoder 侧的计算。

3. Vision Encoder 和 LLM 往往需要不同的并行策略

先看 Pipeline Parallelism。一种常见的放置方式，是将 Vision Encoder、Projector 和一部分 LLM Layer 共置在第一个 Pipeline stage：

```
PP0: [Vision Encoder + Projector + LLM Layer 0..5]
PP1: [LLM Layer 6..17]
PP2: [LLM Layer 18..29]
PP3: [LLM Layer 30..41]
```

对于纯文本 LLM，可以通过调整每个 stage 的 Layer 数，让各 stage 的平均时间尽量接近。但在上面的布局中：

```
T_PP0
= T_encoder(images, resolution, frames)
  + T_projector
  + T_llm_layer_0..5
```

T_PP0 不仅可能高于其他 stage，还会随 micro-batch 中的视觉负载变化。当 PP0 成为瓶颈时，下游 stage 会因为没有可消费的 micro-batch 而空转，并产生额外的 pipeline bubble 或 back-pressure。

再看 TP、CP 和模型放置。一个大型 MoE LLM 可能为了 hidden size、长上下文和 expert 参数数量，使用较大的 TP、CP、PP 和 EP。但几百 M 参数的 Vision Encoder 往往不需要相同程度的切分：

- 过大的 TP 可能让 Encoder 的单卡 GEMM 变小，通信占比反而上升。
- LLM 和 Vision Encoder 的序列长度与显存压力不同，最优 CP degree 往往也不同。
- 目前主流的 ViT 并不使用 MoE，因此 LLM 的 EP 并不适合 ViT。

训练系统因此面临一个取舍（当然，现在似乎已经有一些额外的处理方式，在此我就比较简化地来描述）：

```
方案 A：Encoder 和 LLM 共用一套并行配置
          实现较简单，但 Encoder 可能被过度切分，或使 PP0 过重。

方案 B：Encoder 和 LLM 各用一套并行配置
          两边可分别优化；如果 GPU 分组或张量切分方式不兼容，
          边界处的输出和梯度还需要额外转换数据布局。
```

简而言之，共用并行配置可能损失效率，分开配置则会增加调度和通信复杂度。

4. Encoder-free 解决了什么，又没有解决什么

Encoder-free 的系统价值，是拿掉了深层 ViT，让主要的视觉建模和文本建模复用同一套 decoder stack。从 Infra 角度看，这主要带来三个好处：

- 减少 Encoder 和 LLM 之间的并行配置冲突。
- 减少只有部分样本才执行深层 Encoder 的条件分支。
- 让调度、profiling 和性能建模更接近现有的 LLM Infra。

当然，不同分辨率、图片数量和视频帧数仍然会让不同样本产生不同数量的视觉 token，因此样本之间的变长序列和负载差异不会完全消失。改善的是：这些 token 不再需要先经过一个深层、具有独立并行需求的 ViT。